# Catching SMS Spam with Naive Bayes

**Team members:** Abinash Pant & Prashant Aryal

**GitHub repository:** https://github.com/2abi5/cs5830_project5

**Presentation slides:**
https://docs.google.com/presentation/d/1ZWeJyLG4t8Z6ysD744rQaAkgos07oiSd/edit?usp=sharing&ouid=104804874161619715250&rtpof=true&sd=true

## INTRODUCTION

A spam filter has two jobs that pull against each other: catch the spam, and never hide a real message. We wanted to see how well a Naive Bayes filter does both on text messages, and what the choice between multinomial and Gaussian Naive Bayes changes. Naive Bayes applies Bayes' rule word by word, trains in milliseconds and shows which words drove each decision, so it is a natural first tool. A lost real message costs more than a missed spam.

We used 5,159 unique text messages. The final filter, a Bernoulli Naive Bayes model chosen by cross-validation, caught 112 of 128 spam messages in 1,032 unseen messages and flagged none of the 904 legitimate ones (precision 1.00, recall 0.875, F1 0.933). Multinomial Naive Bayes fits word counts, Gaussian Naive Bayes fits numeric message statistics but fails on word counts, and a hybrid of the two added false alarms. Phone numbers and prize words give spam away, and the spam that slips through reads like ordinary chat.

## DATASET

We used the UCI SMS Spam Collection (Almeida et al., 2011): 5,574 English text messages labelled ham (legitimate) or spam, merged from four public sources. The raw file needed cleaning. We fixed 601 messages with broken characters, such as HTML entities, and removed 415 duplicate copies, because a message in both the training and test sets measures memory, not generalisation. One source also hid numbers and times behind placeholders such as <#> that appear only in ham. Left alone, <#> would be the strongest ham word, so we mapped the placeholders to the same tokens as real numbers. Links, money amounts and phone numbers became category tokens.

That left 5,159 messages, 641 of them spam (12.4%). We built a 7,000-word bag-of-words vocabulary and eight numeric statistics per message (characters, words, mean word length, digits, uppercase share, exclamation marks, currency symbols and links). A stratified 80/20 split gave 4,127 training and 1,032 test messages. Spam differs visibly from ham: 51% of spam is 140 to 159

characters long, while most ham is under 60 (Figure 1). Spam also has far more digits and capital letters, and most statistics are skewed or piled at zero (Figure 2), which matters for the Gaussian model.

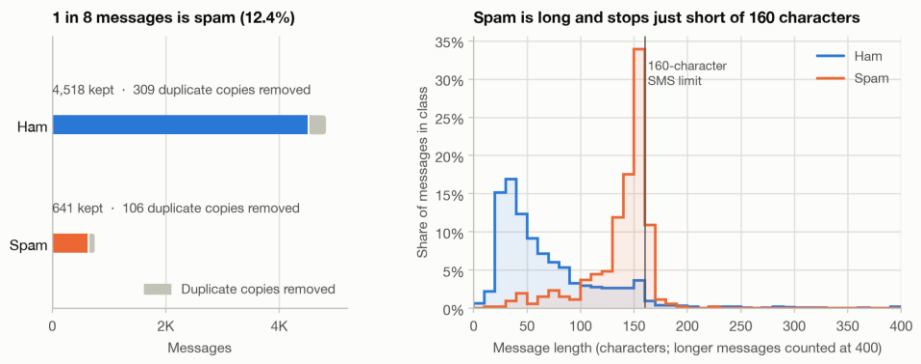



**Figure 1.** Class balance and message length of spam versus ham

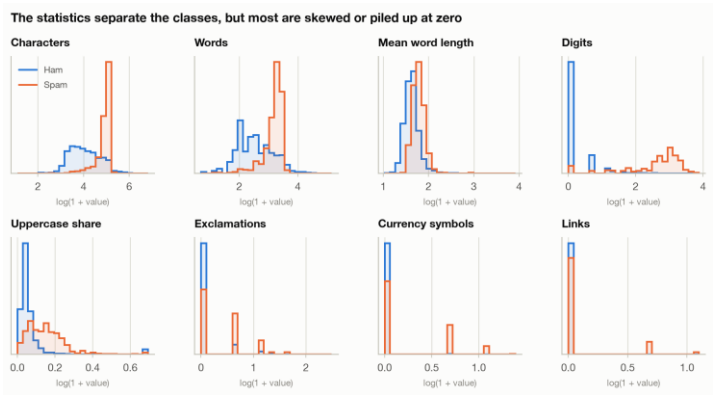



**Figure 2.** Message statistics by class: spam is longer and has more digits.

## ANALYSIS TECHNIQUE

Naive Bayes picks the class with the larger score log P(class) + sum of log P(feature | class), where the naive step assumes features are independent once the class is known. The models differ only in the shape they assume for each feature. Multinomial uses word counts, Bernoulli uses whether a word is present, and Gaussian uses a normal curve for continuous values. We matched each model to the features it assumes: multinomial and Bernoulli on words, Gaussian on the eight message statistics after a log(1 + x) transform. As a check, we also fitted Gaussian to word counts on purpose. The hybrid adds the multinomial and Gaussian log-likelihoods, since words and statistics are independent given the class under the naive assumption. We compared all of these with logistic regression and a majority-class baseline (always "ham").

We tuned smoothing, n-gram range and regularisation with 5-fold cross-validation on the training set only, scored by F1 on the spam class. Accuracy would reward the baseline, which is 87.6% accurate and catches nothing. We then tested each tuned model once and report precision, recall and F1 with a 95% bootstrap interval, plus ROC and precision-recall curves.

## RESULTS

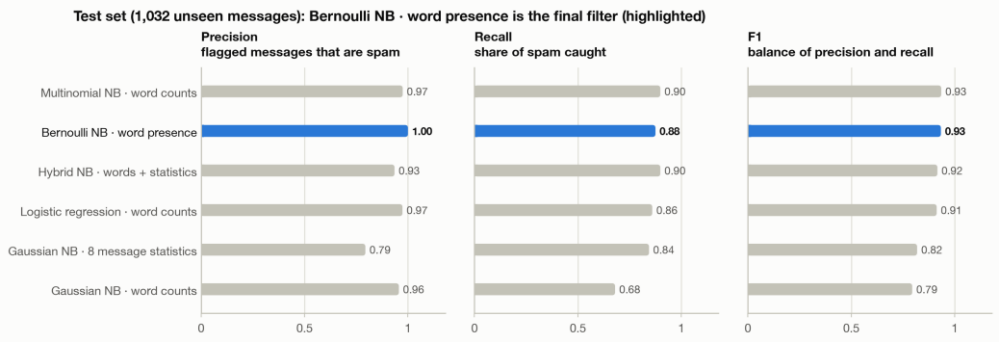



**Figure 3.** Cross-validated and test-set F1 for each model. Bernoulli and multinomial Naive Bayes lead and are statistically tied.

Bernoulli Naive Bayes had the best cross-validated F1 (0.967) and became the final filter. On the test set it caught 112 of 128 spam and flagged no ham. Multinomial was statistically tied (F1 0.935 against 0.933, with overlapping intervals), and both beat logistic regression, which flagged 3 ham messages and missed 18 spam. Gaussian on word counts caught only 87 of 128 spam (recall 0.68). Gaussian on the statistics reached F1 0.818 but flagged 28 ham messages, because long, number-heavy legitimate messages look like spam on length alone.

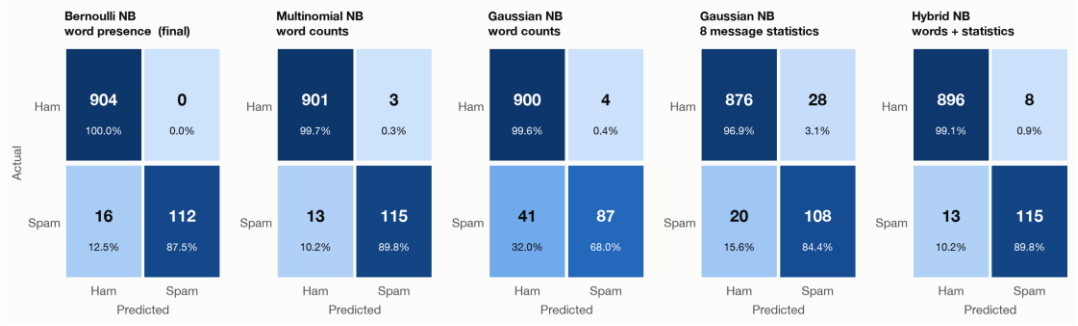



**Figure 4.** Test-set confusion matrices for the five Naive Bayes models. Rows are the true class, columns the prediction.

The hybrid did not help. It caught as much spam as multinomial (115 of 128) but flagged 8 ham messages instead of 3. Characters and words correlate at 0.97 within ham, so the Gaussian part counts message length twice, which is the independence assumption failing in practice.

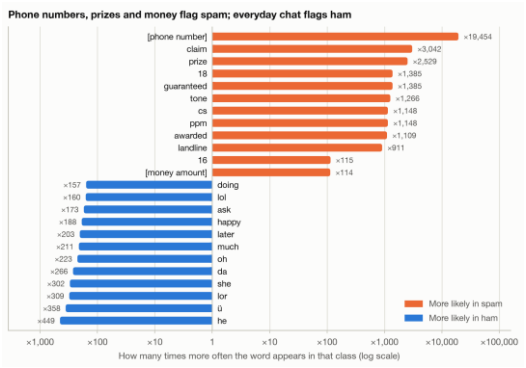



**Figure 5.** Top 12 spam and ham words from multinomial Naive Bayes.

What gives spam away is contact and reward words (a phone number, "claim", "prize", "awarded"), premium-rate jargon ("ppm", "cs", "tone") and age gates ("18"). Ham evidence is everyday chat ("lol", "later") and Singlish particles ("lor", "da"). All 16 test errors were missed spam, and 12 of them contain no phone number, money amount or link. They read like chat, such as a driving joke or a trivia question, so word evidence cannot separate them from ham.

## TECHNICAL

We unescaped HTML entities, mapped Windows-1252 control codes to ASCII and removed duplicates after that fix. Normalising numbers into tokens left cross-validated F1 unchanged (0.955 against 0.956 on raw text), shrank the vocabulary by 8% and lets the model recognise unseen phone numbers. We used scikit-learn 1.9 for the models and cross-validation, and implemented multinomial Naive Bayes from scratch in NumPy; its log-probabilities match scikit-learn to within 1.4 x 10^-14. The random seed is 5830 throughout.

We also checked a cost-based threshold (a false alarm costing 10 times a miss) and kept the default 0.5, because Bernoulli is already conservative. Learning curves show Naive Bayes needs far less data than logistic regression: with 66 training messages it reaches F1 0.77 to 0.84 against 0.67. Limitations: the ham comes mostly from Singapore and the spam from the UK, collected before 2011, so modern scams would need retraining. The test set holds only 128 spam, so F1 intervals are about 0.03 wide and smaller gaps between models are not meaningful. Next time we would retrain on recent messages and add character n-grams to catch spellings like "fr33".